DAY 01 · FOUNDATIONS

Python Foundations + Running Tests

Types, control flow, functions, lists, and the edit-test-fix loop.

2–3 hours / day01_foundations / Core Python

INSIDE THIS BOOKLET

Types | Control flow | Functions | Lists | unittest

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day01_foundations/` · the course runner `python run.py 1`

In this session

Before we start: what the exam actually asks of you 3

1 The REPL: your scratchpad 3

2 Numbers and the surprises they hide 4

3 True, False, and "truthiness" 7

4 Control flow: deciding and repeating 9

5 Functions 12

6 Lists 13

7 Tests: how your work is checked 17

8 The toolbox: everything else the exercises use 19

Exercises 25

End-of-day checklist 26

> **◎ Today's goal**
>
> Write small functions with confidence, know the core types and list operations cold, and get comfortable with the "edit → run tests → read failure → fix" loop you'll use in the exam.

Before we start: what the exam actually asks of you

In the test you will not be asked to invent a program. You'll be given a file of Python code (a hundred lines or a few hundred) that *almost* works. Someone has written a correct program and then changed a handful of small things in it to break it. You'll also get a written description of what the code is supposed to do (the **spec**) and a set of automatic checks (the **tests**). Your score is how many of those checks pass when time runs out. That changes what "knowing Python" means for you. You need to be able to look at a line like

```
mid = (lo + hi) / 2
```

and immediately think "that gives a decimal number, but it's used as a list position, so it should be `//`". The bugs are rarely exotic. They're tiny details: a `<` that should be `<=`, a `return` that's indented one level too far, a list that's shared when it should be copied. Today we go through the basic building blocks of Python slowly, and for each one we'll look at exactly those details. Read this file with a terminal open next to it. Every time you see an example, type it in yourself. Reading code and running code teach you different things, and you need both.

1 The REPL: your scratchpad

1.1 What it is

If you open a terminal and type `python3` (and press Enter), you get a prompt that looks like `>>>`. This is the **REPL**, short for *Read, Evaluate, Print, Loop*: Python reads what you type, evaluates it, prints the result, and waits for the next line. It's a calculator that understands all of Python.

```
$ python3
>>> 2 + 3
5
>>> "hello".upper()
'HELLO'
>>> x = 10
>>> x * 2
20
```

You don't need `print()` here. The REPL prints the value of whatever expression you type. To leave it, type `exit()` or press Ctrl-D.

1.2 Why it matters in the exam

The exam forbids AI and web search, but it does **not** forbid running Python. So whenever you're unsure what a line of the broken code does, you can copy a small version of it into the REPL and see. For example, if the code says `-7 // 2` and you're not sure whether that's `-3` or `-4`, you don't have to guess or reason it out:

```
>>> -7 // 2
-4
```

Five seconds, and you know.

> **REPL habit**
>
> **When unsure how something behaves, try it in the REPL in 5 seconds** instead of guessing. In the exam, a 5-second REPL check beats a 10-minute bug hunt.

1.3 Built-in documentation: `help()` and `dir()`

Python ships with its own documentation, and the REPL can show it. That's your offline manual during the exam. `help(something)` prints the documentation for a function or method: what arguments it takes and what it does.

```
>>> help(list.insert)
Help on method_descriptor:

insert(self, index, object, /)
    Insert object before index.
```

This tells you that `insert` takes a position (`index`) and a value (`object`), and puts the value *before* that position. When the help text is long, it opens in a pager: scroll with the arrow keys and press `q` to get back to the prompt. `dir(something)` lists everything you can do with a value. It's useful when you remember that a method exists but not its exact name.

```
>>> dir("")
[... 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', ... 'isdigit', ...]
```

(The names starting with `__` are internal machinery. You can ignore them for now. We'll meet some on Day 4.) You can also type several expressions separated by commas to compare them side by side. Python packs them into a *tuple* (a fixed group of values, written in parentheses) and prints it:

```
>>> 7 // 2, 7 / 2, -7 // 2, -7 % 3
(3, 3.5, -4, 2)
```

2 Numbers and the surprises they hide

Python has two main kinds of number: **integers** (`int`, whole numbers like `7` or `-3`) and **floating-point numbers** (`float`, numbers with a decimal point like `3.5`). Most arithmetic behaves exactly as you learned at school. A few operations don't, and those are exactly where exam bugs like to hide, because the wrong version looks perfectly reasonable.

2.1 Two kinds of division: `/` and `//`

Python has two division operators, and they do different things. `/` is ordinary division. It **always** gives a float, even when the division is exact:

```
>>> 7 / 2
3.5
>>> 6 / 2
3.0
```

Notice that `6 / 2` gives `3.0`, not `3`. The `.0` means it's a float. `//` is **floor division**: it divides and then rounds *down* to a whole number. With integers, the result is an int.

```
>>> 7 // 2
3
>>> 6 // 2
3
```

Why does this matter? Because positions in a list must be integers. If you try to use a float as a position, Python refuses:

```
>>> letters = ["a", "b", "c", "d"]
>>> letters[4 / 2]
TypeError: list indices must be integers or slices, not float
>>> letters[4 // 2]
'c'
```

So whenever you see a division whose result is used as a position, a size or a count (for example "the middle of the list"), it should almost certainly be `//`. If it's `/`, you've found a bug.

2.2 "Rounding down" with negative numbers

"Round down" means "go toward minus infinity", which is not the same as "drop the decimals" when the number is negative.

```
>>> 7 // 2      # 3.5 rounded down
3
>>> -7 // 2     # -3.5 rounded down: -4 is below -3.5
-4
```

Many people expect `-3` here. There's a different function that *does* just drop the decimals: `int()`. It rounds toward zero:

```
>>> int(3.5)
3
>>> int(-3.5)
-3
```

For positive numbers, `//` and `int(a / b)` give the same answer. For negative numbers they differ. If a spec says "truncate toward zero", it means `int(a / b)`. If it says "floor", it means `a // b`.

2.3 The remainder: `%`

`%` (called "modulo") gives the remainder of a division:

```
>>> 17 % 5      # 17 = 3*5 + 2
2
>>> 10 % 2      # even numbers have remainder 0
0
```

It's used constantly: `n % 2 == 0` tests whether a number is even, and `i % len(items)` makes an index "wrap around" to the start of a list when it goes past the end. With negative numbers, Python's `%` always gives a result with the same sign as the number on the right:

```
>>> -7 % 3
2
```

This is actually convenient. It means `(i - 1) % len(items)` correctly wraps from position 0 back to the last position, instead of producing a negative number.

2.4 Rounding: `round()`

`round()` rounds to the nearest integer, but when a number is exactly halfway (`.5`) it goes to the nearest **even** number. This is called "banker's rounding":

```
>>> round(2.5)
2
>>> round(3.5)
4
>>> round(2.6)
3
```

You probably won't need this often, but if a test expects `3` and the code uses `round(2.5)`, now you know why it gets `2`.

2.5 Floats are approximate

Computers store floats in binary, and most decimal fractions (like `0.1`) can't be represented exactly in binary, just as `1/3` can't be written exactly in decimal (`0.3333...`). The tiny errors usually don't show, but they're there:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

The lesson: **never compare floats with `==`**. Instead, check whether they're *close enough*, i.e. whether the difference between them is tiny:

```
>>> abs((0.1 + 0.2) - 0.3) < 1e-9
True
```

(`abs` gives the absolute value, which is the distance from zero, and `1e-9` is scientific notation for 0.000000001.) Tests do the same thing using a special check called `assertAlmostEqual`, which we'll see in section 7. When money is involved, careful code avoids floats altogether and counts in whole cents instead (`1999` cents rather than `19.99` euros).

3 True, False, and “truthiness”

3.1 Booleans

A **boolean** (`bool`) is a value that is either `True` or `False`. Comparisons produce booleans:

```
>>> 3 > 2
True
>>> 3 == 4
False
>>> "a" != "b"
True
```

`if` and `while` use booleans to decide what to do:

```
if temperature > 30:
    print("hot")
```

3.2 Every value can act as true or false

Here’s something Python does that many languages don’t: you can put **any** value after `if`, not just a boolean. Python decides whether it counts as true or false. This is called the value’s **truthiness**. You can see the decision Python makes by calling `bool()` on the value:

```
>>> bool(5)
True
>>> bool(0)
False
>>> bool("hello")
True
>>> bool("")
False
>>> bool([1, 2])
True
>>> bool([])
False
>>> bool(None)
False
```

The rule is simple. These values count as false, and we call them **falsy**:

- the number zero (`0` or `0.0`)
- an empty string `""`
- an empty list `[]`, empty dict `{}`, empty set or tuple
- the special value `None` (which means “nothing here”; more on it below)

Everything else counts as true (**truthy**). This lets you write short, readable checks like:

```
if items:              # "if the list is not empty"
    print(items[0])

if not name:           # "if the name is empty"
    name = "anonymous"
```

3.3 What is None?

None is a special value that means "no value". A function that doesn't return anything gives back None. Dict lookups that find nothing can give back None (Day 3). Code often uses None to mean "not set yet" or "not found". To check whether something is None, write `x is None` (not `x == None`). Both usually work, but `is` checks "is this the very same object", and since there is only one None in the whole program, that's the precise question. `is None` is also what every experienced Python programmer writes, so you'll see it everywhere.

3.4 The trap: when zero is a real answer

Truthiness is convenient, but it treats "zero", "empty" and "nothing" as the same thing, and sometimes they aren't.

Truthiness trap

`If not x:` is True for `0`, `""`, `[]`, None. If `0` is a valid value, write `if x is None:` instead. This is a very common exam bug.

Let's see it with a concrete example. Imagine a dictionary that stores how many items of each fruit are left in stock. (A dictionary maps names to values, and Day 3 is entirely about them. For now, `stock.get(name)` means "look up `name` in `stock`, and give back None if it isn't there".)

```
stock = {"apples": 5, "pears": 0}

def describe(name):
    count = stock.get(name)
    if not count:
        return "we don't sell that"
    return f"{count} left"
```

Let's try it:

```
>>> describe("apples")
'5 left'
>>> describe("bananas")
"we don't sell that"
>>> describe("pears")
"we don't sell that"
```

The last answer is wrong. We *do* sell pears, we just have none left. What happened? `stock.get("pears")` returned `0`. Then `if not count:` asked "is `count` falsy?", and `0` is falsy, so the function took the "we don't sell that" branch, exactly as it did for bananas where `count` was None. The author meant "if the fruit isn't in the dictionary", but wrote "if the count is zero or missing". The fix is to ask the precise question:

```
if count is None:          # only when the lookup found nothing
    return "we don't sell that"
```

Now `describe("pears")` correctly says `'0 left'`. (On Day 3 you'll see an even clearer way to write it: `if name not in stock:`.) Whenever you see `if not x:` or `if x:` in exam code, ask yourself: *could `x` legitimately be `0` or an empty string?* If yes, the check is probably a bug.

4 Control flow: deciding and repeating

"Control flow" means the order in which lines run: `if` chooses between alternatives, and loops repeat things. This is where most off-by-one mistakes live, so it's worth knowing *exactly* which values every loop goes through.

4.1 `for` loops and `range`

A `for` loop runs its body once for each item in a sequence:

```
for fruit in ["apple", "pear", "kiwi"]:
    print(fruit)
```

To loop over numbers, use `range`. The key fact about `range`: **it stops before the end number**. `range(5)` gives 0, 1, 2, 3, 4 (five numbers, but not 5).

```
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> list(range(2, 6))        # start at 2, stop before 6
[2, 3, 4, 5]
>>> list(range(2, 10, 3))    # count in steps of 3
[2, 5, 8]
>>> list(range(5, 0, -1))    # count down; still stops BEFORE 0
[5, 4, 3, 2, 1]
```

(`list(...)` just turns the range into a list so the REPL can show all the numbers at once.) Why does `range` exclude the end? Because then `range(len(items))` produces exactly the valid positions of a list. A list with 4 items has positions 0, 1, 2 and 3, and `range(4)` is exactly 0, 1, 2, 3. This is also where many bugs hide. Whenever you see a `range` in exam code, ask: *should the last value be included?*

- "for each position in the list" → `range(len(items))`
- "for each pair of neighbours `items[i]`, `items[i+1]`" → `range(len(items) - 1)`, because the last position has no neighbour after it
- "from 1 to n, including n" → `range(1, n + 1)`
- "count down from n–1 to 0, including 0" → `range(n - 1, -1, -1)`

4.2 Getting the position and the value together: `enumerate`

Sometimes you need both the position of an item and the item itself. `enumerate` gives you both:

```
>>> for i, fruit in enumerate(["apple", "pear", "kiwi"]):
...     print(i, fruit)
...
0 apple
1 pear
2 kiwi
```

(The `...` is the REPL telling you it's waiting for the rest of the loop. Press Enter on an empty line to run it.) Positions start at 0. If you need them to start at 1, for example for line numbers in an error message ("error on line 1"), write `enumerate(lines, start=1)`. Forgetting `start=1` is a classic exam bug when a spec talks about "line numbers".

4.3 Walking two lists together: `zip`

Imagine two lists that belong together: a list of names and a list of ages, where position 0 of one goes with position 0 of the other. You could loop over the positions with `range(len(names))` and index both lists. `zip` does that pairing for you, like a zip fastener joining its two sides tooth by tooth. `zip(a, b)` produces **pairs**: the first item of `a` with the first item of `b`, then the second with the second, and so on. Each pair is a **tuple**, a small fixed group of values written in parentheses (more on tuples in section 8). To *see* the pairs in the REPL, wrap the call in `list(...)`, because `zip` on its own produces its pairs one at a time, only when something asks for them:

```
>>> names = ["ana", "ben", "cai"]
>>> ages = [31, 25, 40]
>>> zip(names, ages)
<zip object at 0x7f...>            # not the pairs themselves, just a "pair maker"
>>> list(zip(names, ages))
[('ana', 31), ('ben', 25), ('cai', 40)]
```

Most of the time you use it directly in a `for` loop. Because each item is a pair, you can give the loop **two** variables, and Python splits each pair into them (this is called *unpacking*, also in section 8):

```
>>> for name, age in zip(names, ages):
...     print(name, "is", age)
...
ana is 31
ben is 25
cai is 40
```

Round by round: the first pair is `('ana', 31)`, so `name = 'ana'` and `age = 31`; the second is `('ben', 25)`, and so on. That is exactly the same as this longer version, which you should be able to translate back and forth in your head:

```
for i in range(len(names)):
    name, age = names[i], ages[i]
    print(name, "is", age)
```

Three more facts about `zip`:

- It can take more than two lists: `zip(a, b, c)` gives triples.
- **If the lists have different lengths, it stops quietly at the end of the shorter one.** No error, the extra items are simply ignored. `list(zip([1, 2, 3], ["x"]))` is `[(1, 'x')]`. That's convenient, but it can also hide a bug.
- Like `range`, a zip object can only be walked through once. If you need the pairs twice, store them with `list(...)`.

The neighbours trick. A very common need is to look at each item *together with the next one*: to compute differences, to check that a list is sorted, to find where values jump. `zip` does it neatly when you zip a list with itself shifted by one:

```
>>> nums = [1, 4, 9, 16]
>>> nums[1:]                       # the list without its first item (slicing, section 6)
[4, 9, 16]
>>> list(zip(nums, nums[1:]))
[(1, 4), (4, 9), (9, 16)]
```

Line the two lists up to see why it works:

```
nums      :  1   4   9   16
nums[1:]  :  4   9   16
pairs     : (1,4) (4,9) (9,16)     # zip stops when nums[1:] runs out
```

Each column is one item and the item after it. The last item, 16, has no partner, and zip's "stop at the shorter list" rule drops it, which is exactly what we want. So "the difference between each number and the next" becomes:

```
[b - a for a, b in zip(nums, nums[1:])]      # [3, 5, 7]
```

(This is a list comprehension, section 6: "for each pair `(a, b)`, compute `b - a`".) If `nums` has 0 or 1 items, `nums[1:]` is empty, zip produces no pairs, and the result is `[]` with no special case needed.

4.4 `while` loops

A `while` loop repeats as long as its condition is true. Use it when you don't know in advance how many steps you'll need, for example when walking through a string and consuming a different number of characters each time.

```
i = 0
while i < len(text):
    ...          # look at text[i], decide how far to move
    i += 1       # MUST move forward, or the loop never ends
```

The danger with `while` is the **infinite loop**: if some path through the body forgets to change the variable that the condition depends on, the condition stays true forever and the program hangs. In the exam, a test that never finishes almost always means this. For every `while` loop you read, check that *every* path through its body makes progress.

4.5 `break` and `continue`

Inside any loop:

- `break` leaves the loop immediately.
- `continue` skips the rest of the current round and goes on to the next one.

```
for n in numbers:
    if n < 0:
        continue      # ignore negatives, go to the next number
    if n == 0:
        break         # stop the whole loop at the first zero
    print(n)
```

4.6 The `for ... else` oddity

Python lets you attach an `else` to a loop. It runs **only if the loop finished without hitting `break`**. It's a way of saying "if we searched everything and never found it":

```
for x in items:
    if x == target:
        print("found it")
        break
else:
    print("not found")
```

It's rare, but it confuses people who assume the `else` belongs to the `if` inside the loop. Look at the indentation: this `else` lines up with `for`, not with `if`.

5 Functions

5.1 Defining and calling

A **function** is a named, reusable piece of code. It receives inputs (**parameters**), does some work, and hands back a result with `return`.

```
def area(width, height):
    return width * height
```

```
>>> area(3, 4)
12
```

In the exam, almost every function has a **docstring**: a string in triple quotes right under the `def` line that describes what the function must do. That docstring is the spec for that function, and your job is to make the body match it:

```
def area(width, height):
    """Return the area of a width x height rectangle. Raise ValueError if either side is
       negative."""
```

5.2 Kinds of parameters

You'll see a few ways of declaring parameters. Here they all are in one signature:

```
def f(a, b=10, *args, key=None, **kwargs):
    ...
```

- `a` is an ordinary parameter. The caller must provide it.
- `b=10` has a **default value**. If the caller doesn't provide `b`, it's 10.
- `*args` collects any *extra* positional values into a tuple.
- `key=None` comes after `*args`, which makes it **keyword-only**: it can only be passed by name, as in `f(1, key="x")`.
- `**kwargs` collects any extra named values into a dictionary.

Here's how calls map to those parameters:

```
f(1)            -> a=1, b=10
f(1, 2)         -> a=1, b=2
f(1, 2, 3, 4)   -> a=1, b=2, args=(3, 4)
f(1, key="x")   -> a=1, b=10, key="x"
```

In practice you'll mostly see the first two kinds. The others you just need to recognise.

5.3 Returning several values

A function can return several values separated by commas. Python packs them into a tuple, and the caller can unpack them into separate variables:

```
def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([4, 1, 9])    # low = 1, high = 9
```

The built-in `divmod` works the same way: `q, r = divmod(17, 5)` gives `q = 3`, `r = 2`.

5.4 What a function returns when you forget `return`

Every function returns *something*. If execution reaches the end of the body without meeting a `return`, the function returns `None`. That leads to two of the most common bugs you'll meet.

Missing return

A function that builds a result but forgets `return` returns `None`. Test output will say something like `AssertionError: None != [1, 2, 3]`.

```
def doubled(nums):
    result = []
    for n in nums:
        result.append(n * 2)
    # forgot: return result
```

```
>>> print(doubled([1, 2, 3]))
None
```

The function did all the work and then threw it away. When a failing test shows `None` where it expected a real value, check the `return` statements first: is one missing, or is it inside an `if` that wasn't taken?

Return inside the loop

Returns after the first iteration.

```
def total(nums):
    s = 0
    for n in nums:
        s += n
        return s     # BUG: indented one level too deep
```

In Python, indentation isn't decoration. It decides which lines belong to which block. Here `return s` is indented at the same level as `s += n`, so it's part of the loop body. The function adds the first number and returns immediately:

```
>>> total([1, 2, 3])
1
```

Moving `return s` four spaces to the left puts it after the loop, and the function returns `6`. A useful clue: if a function gives the right answer for one-item inputs but the wrong answer for longer ones, suspect an indentation problem.

6 Lists

A **list** is an ordered collection of values, written with square brackets: `[3, 1, 2]`. Lists can grow and shrink, and you can change their contents. (Programmers say they're *mutable*.) They're the most common data structure in exam code: they hold the input, collect the results, and serve as stacks and queues.

6.1 Changing a list in place vs. getting a new value

The most important thing to understand about list methods is this: **some methods change the list itself and return `None`; others leave the list alone and return a new value.** Confusing the two is a frequent bug. Let's go through the main ones in the REPL:

```
>>> a = [3, 1, 2]
>>> a.append(4)          # add one item at the end
>>> a
[3, 1, 2, 4]
>>> a.extend([5, 6])     # add every item of another list
>>> a
[3, 1, 2, 4, 5, 6]
>>> a.insert(0, 9)       # put 9 at position 0, shifting the rest right
>>> a
[9, 3, 1, 2, 4, 5, 6]
```

Notice that `a.append(4)` printed nothing. It changed `a` and returned `None`. A related mistake is `a.append([5, 6])`: that adds *one* item, which happens to be a list, giving `[..., [5, 6]]`. `extend` is the one that adds each element. Removing items:

```
>>> a.pop()              # remove the LAST item and give it back
6
>>> a.pop(0)             # remove the item at position 0 and give it back
9
>>> a
[3, 1, 2, 4, 5]
>>> a.remove(2)          # remove the first item EQUAL to 2 (not position 2!)
>>> a
[3, 1, 4, 5]
>>> a.index(4)           # where is the first 4?
2
```

`remove` and `index` raise an error (`ValueError`) if the value isn't in the list. `append` together with `pop()` lets you use a list as a **stack**: the last thing you put in is the first thing you take out, like a pile of plates. You'll see stacks in parsers and in depth-first search.

6.2 Sorting: `sort()` vs `sorted()`

This pair is the classic example of "changes in place" vs "returns a new value".

```
>>> a = [3, 1, 2]
>>> b = sorted(a)        # returns a NEW sorted list; a is untouched
>>> b
[1, 2, 3]
>>> a
[3, 1, 2]
>>> a.sort()             # sorts a itself, returns None
>>> a
[1, 2, 3]
```

The trap is writing `a = a.sort()`. That sorts the list and then replaces `a` with the return value of `sort()`, which is `None`. Your list is gone. Both accept a `key` argument, which is a function that says *what to sort by*. A `lambda` is a quick way to write a small one-line function inline: `lambda w: len(w)` means "given `w`, return `len(w)`".

```
>>> words = ["kiwi", "fig", "banana", "apple"]
>>> sorted(words)                            # alphabetical
['apple', 'banana', 'fig', 'kiwi']
>>> sorted(words, key=lambda w: len(w))      # by length
['fig', 'kiwi', 'apple', 'banana']
>>> sorted(words, key=lambda w: (len(w), w)) # by length, then alphabetically for equal
    lengths
['fig', 'kiwi', 'apple', 'banana']
>>> sorted([3, 1, 2], reverse=True)          # biggest first
[3, 2, 1]
```

The third example uses a **tuple** as the key. Python compares tuples item by item: first by length, and only if the lengths are equal, by the word itself. That's how you implement a spec that says "sort by X, and break ties by Y". We'll use this a lot on Day 3.

6.3 Slicing: taking a piece of a list

A **slice** gives you a part of a list, written `a[start:stop]`. Like `range`, it includes `start` and **stops before** `stop`. Negative numbers count from the end: `-1` is the last item, `-2` the one before.

```
>>> a = [0, 1, 2, 3, 4, 5]
>>> a[1:4]       # positions 1, 2, 3
[1, 2, 3]
>>> a[:2]        # from the start: the first 2 items
[0, 1]
>>> a[-2:]       # the last 2 items
[4, 5]
>>> a[::2]       # every second item (the third number is the step)
[0, 2, 4]
>>> a[::-1]      # step -1: a reversed copy
[5, 4, 3, 2, 1, 0]
```

A useful way to picture it is that the numbers point at the *gaps between* items:

```
| 0 | 1 | 2 | 3 | 4 | 5 |
0   1   2   3   4   5   6
```

`a[1:4]` takes everything between gap 1 and gap 4, which is three items. In general `a[i:j]` contains `j - i` items. Two more facts:

- Asking for a single position that doesn't exist (`a[10]`) is an error (`IndexError`), but a slice that goes too far just gives you fewer items, or none: `a[10:]` is `[]`. Slices never raise.
- The same slicing works on strings (Day 2) and bytes (Day 5).

There's one subtle trap. To get "the last `n` items" you'd naturally write `a[-n:]`. But look at what happens when `n` is 0:

```
>>> n = 0
>>> a[-n:]
[0, 1, 2, 3, 4, 5]
```

You wanted zero items and got the whole list. That's because `-0` is just `0`, so `a[-0:]` is `a[0:]`, "everything from the start". The safe version is `a[len(a) - n:]`. Tests love checking the `n = 0` case.

6.4 Two names, one list: aliasing

This is one of the most important ideas in Python, so take your time with it. In Python, a variable isn't a box that holds a value. It's a **name tag attached to a value**. When you write `b = a` with a list, you don't copy the list. You attach a second name tag to the *same* list:

```
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]
```

We changed `b`, and `a` changed too, because they're the same list. This is called **aliasing**. If you want an independent copy, you have to ask for one:

```
>>> a = [1, 2, 3]
>>> b = a[:]          # or list(a), or a.copy()
>>> b.append(4)
>>> a
[1, 2, 3]
```

Now `b` is a separate list. When a spec says "returns a new list" or "must not modify its input", check that the code really makes a copy. The same issue shows up in a sneakier form when building a grid (a list of lists):

```
>>> grid = [[0] * 3] * 3
>>> grid
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
>>> grid[0][0] = 1
>>> grid
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

We set one cell, and a whole column changed. `[0] * 3` builds one row, and `* 3` then makes a list containing that *same row* three times: three name tags for one list. The correct way builds a fresh row each time:

```
grid = [[0] * 3 for _ in range(3)]
```

(This uses a *comprehension*, explained next. `_` is a conventional name for a loop variable you don't use.) You'll need this when building tables in the dynamic-programming drills.

6.5 List comprehensions

A **list comprehension** builds a new list from a loop, in one line. This:

```
squares = []
for x in range(5):
    squares.append(x * x)
```

can be written as:

```
squares = [x * x for x in range(5)]       # [0, 1, 4, 9, 16]
```

Read it as "a list of `x * x`, for each `x` in `range(5)`". You can add a condition to keep only some items:

```
evens = [x for x in nums if x % 2 == 0]   # only the even numbers
```

And you can have two `for`s, which work like nested loops, read left to right (the first `for` is the outer loop):

```
flat = [x for row in grid for x in row]   # every x of every row: flattens a grid
```

6.6 Don't remove from a list while looping over it

Here's a bug that looks completely reasonable:

```
nums = [-1, -2, 3]
for x in nums:
    if x < 0:
        nums.remove(x)
print(nums)          # [-2, 3]  -- the -2 survived!
```

Why? The `for` loop keeps track of its position by number. In the first round it's at position 0 and sees `-1`, which gets removed. Everything shifts left: `-2` moves to position 0 and `3` to position 1. The loop then moves on to position 1, which is now `3`. It never looks at `-2`. The fix is not to change the list you're looping over. Build a new list with just the items you want to keep:

```
nums = [x for x in nums if x >= 0]
```

7 Tests: how your work is checked

7.1 What a test is

The exam grades you with Python's built-in testing tool, **unittest**. A test is a small piece of code that calls your function with a known input and checks that the output is what it should be. Here's a complete test file:

```
import unittest
from ex01_lists import running_total

class TestRunningTotal(unittest.TestCase):
    def test_basic(self):
        self.assertEqual(running_total([1, 2, 3]), [1, 3, 6])

    def test_empty(self):
        self.assertEqual(running_total([]), [])

if __name__ == "__main__":
    unittest.main()
```

Piece by piece:

- `import unittest` loads the testing tool, and `from ex01_lists import running_total` loads the function being tested.
- `class TestRunningTotal(unittest.TestCase):` creates a group of related tests. (Classes are Day 4's topic. For now, treat this line as a fixed pattern.)
- Each method whose name starts with `test` is one test. unittest finds and runs them automatically.
- `self.assertEqual(actual, expected)` is the check itself: "these two must be equal". If they are, the test passes. If not, it fails and prints both values.
- The last two lines let you run the file directly with `python test_file.py`.

7.2 The most common checks

unittest provides several "assert" methods. Here are the ones you'll see most, with what each one checks:

```
self.assertEqual(a, b)           # a == b
self.assertTrue(x)               # x is truthy
self.assertFalse(x)              # x is falsy
self.assertIsNone(x)             # x is None
self.assertIn(item, container)   # item in container
```

Remember the floats from section 2, where `0.1 + 0.2 == 0.3` is `False`? That's why there's a special check for floats, `assertAlmostEqual`. It passes when the two numbers are equal *after rounding to 7 decimal places*, so tiny binary errors don't matter:

```
def test_average(self):
    result = average([0.1, 0.2])          # 0.15000000000000002 in reality
    self.assertAlmostEqual(result, 0.15)  # passes
    # self.assertEqual(result, 0.15)      # would FAIL because of the tiny error
```

And to check that a function correctly *refuses* bad input by raising an error, tests use `assertRaises` with a `with` block:

```
def test_negative_size(self):
    with self.assertRaises(ValueError):
        chunk([1, 2, 3], -1)
```

This reads as: "the code inside this block must raise a `ValueError`". If it does, the test passes. If it raises nothing, or a different kind of error, the test fails. (Errors and `raise` are covered properly on Day 4.)

7.3 Running the tests

From a terminal in the folder that contains the tests:

```
python -m unittest                 # run every test file whose name starts with "test"
python -m unittest -v              # same, but list each test with ok / FAIL / ERROR
python -m unittest test_ex01_lists.TestRunningTotal.test_basic   # run just one test
```

The last form, `file.Class.method`, is worth practising. In the exam, once you've found a failing test, you'll re-run just that one while you work on it, instead of scrolling through fifty results each time. In this course, `python run.py <day>` does the same job from the course's root folder: it runs that day's tests against your exercise files and prints a score like the exam does.

7.4 Reading a failure

When a test fails, unittest prints a report like this:

```
FAIL: test_basic (test_ex01_lists.TestRunningTotal.test_basic)
Traceback (most recent call last):
  File ".../test_ex01_lists.py", line 7, in test_basic
    self.assertEqual(running_total([1, 2, 3]), [1, 3, 6])
AssertionError: Lists differ: [1, 2, 3] != [1, 3, 6]
```

Read it from top to bottom:

1. **`FAIL: test_basic`** tells you which test failed.

2. The **Traceback** tells you which line of the test was running: line 7, which called `running_total([1, 2, 3])`.
3. The last line shows **what your code returned** (first, `[1, 2, 3]`) and **what was expected** (second, `[1, 3, 6]`).

So without opening the function at all, we already know a lot. Given `[1, 2, 3]`, it returned its input unchanged, when it should have returned the running sums (1, then 1+2=3, then 1+2+3=6). A good guess is that it appends each number instead of the sum so far. Now you open the code with a specific suspicion. There are two kinds of bad result, and they're labelled differently:

- **FAIL** means the code ran but gave a wrong answer: an assertion was false.
- **ERROR** means the code crashed with an exception (`TypeError`, `IndexError`, ...). The traceback then continues into *your* code and shows the exact line where it crashed, which makes errors usually quicker to fix.

8 The toolbox: everything else the exercises use

So far we've covered the big ideas. The exercises also lean on a set of small tools that we haven't properly met yet. Some of them get a whole day later on (strings on Day 2, sets on Day 3), but you need the basics today, so here they are, one at a time. Keep the REPL open and try things as you go. If you change an example and it surprises you, that's the good part.

8.1 Adding to a variable: +=

You'll often want to take a variable and change it a little: add 1 to a counter, add a number to a total. You could write `total = total + n`, but this comes up so often that Python has a shorter way: `total += n`. It means exactly the same thing. Here's the classic use, a running total. We start at zero and add each number as we reach it:

```
total = 0
for n in [1, 2, 3]:
    total += n      # total goes 1, then 3, then 6
```

The same shortcut works with the other operators, so you'll also see `i -= 1`, `n *= 2` and `k %= 5`.

8.2 Tuples, and splitting them apart

Sometimes a few values belong together, like the x and y of a point on a map. Python can hold them as one thing, called a **tuple**: `point = (3, 4)`. It looks like a list with round brackets, and the main difference is that you can't change a tuple once it exists. The nice part is getting the values back out. You could write `x = point[0]` and `y = point[1]`, but Python lets you do it in one go by putting two names on the left:

```
>>> point = (3, 4)
>>> x, y = point
>>> x
3
>>> y
4
```

This is called **unpacking**. You need one name per value, or Python complains with a `ValueError`. Once you know this, a few lines you'd otherwise find puzzling make sense:

```
out, s = [], 0          # two variables in one line: out is [] and s is 0
a, b = b, a             # swaps a and b
```

The swap works because Python works out the whole right side first, and only then assigns. So a and b swap without one overwriting the other. Now and then you need to repeat something but don't care about the counter. Programmers name that variable _, a single underscore. It's an ordinary name. It just tells whoever reads the code "I'm not going to use this":

```
for _ in range(3):      # do this 3 times
    print("hi")
```

8.3 Choosing between two values on one line

Say you want a label that says whether a number is even or odd. The long way takes four lines:

```
if n % 2 == 0:
    label = "even"
else:
    label = "odd"
```

When all you're doing is picking between two values, you can write it on one line, almost as you'd say it out loud:

```
label = "even" if n % 2 == 0 else "odd"
```

The order takes a moment to get used to, because the condition sits in the middle: first the value you want if it's true, then the test, then the value you want if it's false. You'll often see this right after a `return`. For example, `return items[0] if items else None` gives back the first item, or `None` if the list is empty.

8.4 Checking that a number is in a range

To check that `base` is between 2 and 16, you could write `2 <= base and base <= 16`. Python lets you write it the way you would in a maths class:

```
if 2 <= base <= 16:
```

Pay attention to `<` versus `<=` here. "From 1 to 10, including both" is `1 <= x <= 10`. If someone writes `<` by mistake, 1 and 10 stop counting, and that's a very common planted bug.

8.5 Asking a list about itself: `len`, `sum`, `min`, `max`

These four do what their names say:

```
>>> nums = [4, 1, 9]
>>> len(nums)          # how many items?
3
>>> sum(nums)          # what do they add up to?
14
>>> min(nums)          # the smallest
1
>>> max(nums)          # the biggest
9
```

One thing can catch you out. `min` and `max` crash with a `ValueError` on an empty list, because an empty list has no smallest item. `sum([])` is fine, and gives 0. So if the list might be empty, check that before you call `min` or `max`. You can also give `min` and `max` separate values instead of a list: `min(3, 7)` is 3 and `max(3, 7)` is 7. Together they can force a number to stay inside a range, which is called **clamping**. Imagine a volume control that only goes from 0 to

1. Whatever number someone asks for, you want the nearest one that's allowed:

```
max(0, min(x, 10))
```

Work from the inside out. `min(x, 10)` stops the number going above 10, and then `max(0, ...)` stops it going below 0. Try it with `x` as -5, 5 and 15, and you'll get 0, 5 and 10.

8.6 Removing the sign: `abs`

`abs(x)` gives you the number without its minus sign, so `abs(-7)` and `abs(7)` are both 7. It's handy when a problem says "ignore the sign", or when it's easier to work on a positive number and put the minus back at the end.

8.7 Dividing and keeping the remainder: `divmod`

You already know `//` (how many times does it fit?) and `%` (what's left over?). When you need both answers, `divmod` gives you the two at once, and you can unpack them:

```
>>> divmod(17, 5)
(3, 2)
>>> q, r = divmod(17, 5)      # 5 fits 3 times into 17, with 2 left over
```

Where this really helps is taking a number apart, one digit at a time. Dividing by 10 chops off the last digit, and the remainder is the digit you chopped off:

```
>>> divmod(472, 10)
(47, 2)
>>> divmod(47, 10)
(4, 7)
>>> divmod(4, 10)
(0, 4)
```

You got the digits 2, 7 and 4, from the last to the first, and you stop when the number reaches 0. The nice surprise is that this works in any base. Use `divmod(n, 2)` to get binary digits, or `divmod(n, 16)` to get hexadecimal ones. You'll need this for `int_to_base`. Do it on a positive number, though, because `//` rounds negative numbers the unexpected way (section 2). That's another job for `abs`.

8.8 Numbers and text are different things

To Python, `42` is a number and `"42"` is text: the character 4 followed by the character 2. They behave very differently. `4 + 2` is 6, but `"4" + "2"` is `"42"`. To go from one to the other, use `str` and `int`:

```
>>> str(42)
'42'
>>> int("42")
42
>>> int("4x")
ValueError: invalid literal for int() with base 10: '4x'
```

Here's a handy trick that comes from this. A `for` loop over a string visits one character at a time, so turning a number into a string is another way to get at its digits:

```
>>> for c in str(305):
...     print(c)
...
3
0
5
```

Each `c` is a one-character string, so if you want to do arithmetic with it, turn it back into a number with `int(c)`.

8.9 Just enough about strings

Day 2 is all about strings, but four small tools show up today, so here's a preview. `"Hello".lower()` gives you a lowercase copy, `'hello'`. This is how you compare text without caring about capitals. `in` asks whether one piece of text appears inside another, so `"e" in "hello"` is `True`. `split` cuts a string into a list of pieces, wherever it finds the separator you give it:

```
>>> "hello big world".split(" ")
['hello', 'big', 'world']
```

`join` does the opposite, gluing a list of strings back into one. Its form looks backwards at first, because you call it on the glue and pass it the pieces:

```
>>> " ".join(["world", "big", "hello"])
'world big hello'
>>> "-".join(["a", "b"])
'a-b'
>>> "".join(["a", "b"])
'ab'
```

Strings can also be indexed and sliced like lists, so `"hello"[0]` is `"h"` and `"hello"[::-1]` is `"olleh"`. Notice that this flips the **letters**. If you want to flip the order of the **words**, split the string into a list first, reverse the list, and join it back together.

8.10 Just enough about sets

Imagine you're at the door of a party with a guest list, checking people off as they arrive. You don't care what order they came in, you can't tick the same person twice, and all you ever ask is "is this person already here?". That's a **set**: a collection with no duplicates and no particular order. You make one with `set()`, and you add to it with `.add()`. A list has `.append()` because it has an end to append to. A set has no end, so the method has a different name.

```
>>> seen = set()
>>> seen.add(7)
>>> seen.add(7)       # it's already there, so nothing happens
>>> seen
{7}
>>> 7 in seen
True
>>> 8 in seen
False
```

A warning: `{}` makes an empty dictionary, not an empty set. To get an empty set you have to write `set()`. If you hand a list to `set()`, you get one copy of each value, which makes it a quick way to throw away duplicates:

```
>>> set([3, 5, 5, 1])
{1, 3, 5}
```

But the order is gone, since sets don't have one. So if you need the values in order, sort them afterwards. `sorted` always gives you back a list, smallest first:

```
>>> sorted(set([3, 5, 5, 1]))
[1, 3, 5]
```

That's why `sorted(set(nums))` is such a common line. It means "the different values in `nums`, from smallest to biggest". The biggest is then at position `-1`, and the second biggest at `-2`, as long as there are at least two values. The other big use is remembering what you've already come across. Suppose you want to remove duplicates from a list but keep the original order. A set alone can't do that, because it forgets the order. So you use two things at once: a list that builds the answer, and a set that remembers what you've seen.

```
seen = set()
out = []
for x in items:
    if x not in seen:
        seen.add(x)
        out.append(x)
```

Let's follow it through with `items = [3, 1, 3]`:

- First comes 3. We haven't seen it, so we note it in `seen` and add it to `out`. Now `out` is `[3]`.
- Then comes 1. It's new as well, so the same thing happens. Now `out` is `[3, 1]`.
- Then 3 comes again. This time it's already in `seen`, so we skip it. `out` stays `[3, 1]`.

Why not just check `if x not in out`? That would give the right answer too, but `in` on a list has to look through the list item by item, while a set can answer straight away, however big it gets. With a long list the difference is huge.

8.11 Going backwards: `reversed`

`reversed(items)` goes through a list from the end to the start, and leaves the original list alone. Like `zip`, it hands out its items one at a time, so to see them all at once, wrap it in `list(...)`:

```
>>> list(reversed([1, 2, 3]))
[3, 2, 1]
```

You'll often pair it with `join`. In `int_to_base`, the digits come out last-to-first, so you collect them in a list, then flip them and glue them together with `"".join(reversed(out))`. Lists also have a method called `items.reverse()`, and it's easy to mix the two up. It's the same story as `sorted` and `sort`: the method changes the list itself and returns `None`.

8.12 Counting with `sum(1 for ...)`

You know that a list comprehension builds a list, as in `[x * x for x in nums]`. If you only want to pass those values straight into a function like `sum`, you can leave out the square brackets:

```
sum(x * x for x in nums)
```

This is called a **generator expression**, but you can think of it as a comprehension with no list at the end of it. It gives us a neat way to count things. How many vowels does a string have?

```
sum(1 for c in s if c in "aeiou")
```

Go through it as Python does. It looks at each character `c`. Whenever `c` is a vowel, it produces a `1`. Then `sum` adds up all those 1s, and the total is the number of vowels. Change the condition, and you can count anything you like. And here's the sum of a number's digits, using the string trick from earlier:

```
sum(int(c) for c in str(472))      # 4 + 7 + 2 = 13
```

8.13 Stopping with an error: `raise`

Sometimes a function is handed something it can't work with, like being asked to split a list into chunks of size -1. Returning a made-up answer would only hide the problem. Instead, the function stops and complains:

```
def chunk(items, size):
    if size <= 0:
        raise ValueError("size must be positive")
    ...
```

`raise` stops the function on the spot. Nothing after it runs, and nothing is returned. The error travels back to whoever called the function. The text in the brackets is a message for the person reading it. Tests normally check only the type of error, with `assertRaises(ValueError)` (section 7), not the wording. Day 4 goes further into errors. You've also seen `raise NotImplementedError` in every exercise you haven't done yet. That's just a placeholder for "nobody has written this yet", so the function fails loudly rather than quietly returning `None`. **Delete that line** and write your code in its place.

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 1` after every change; peek at `solutions/` only after 15 minutes stuck.

There are two kinds of exercise in this course:

- **Implement**: the function's body is empty and its docstring describes what it must do. You write the body. These build fluency.
- **Debug** (files named `*_fix_me.py`): the function is complete but contains exactly one planted bug. These train the exam skill directly.

File	Type	What
`exercises/ex01_lists.py`	implement	7 list functions
`exercises/ex02_numbers.py`	implement	5 number functions (incl. base conversion)
`exercises/ex03_fix_me.py`	**debug**	7 functions that each contain exactly one bug

Which tools each function needs (all of them are explained above; the section is in brackets):

Function	Tools
`running_total`	a loop, `+=` (8), `append` (6)
`chunk`	`range` with a step (4), slicing (6), `raise` (8)
`flatten`	a comprehension with two `for`s (6), or two nested loops
`second_largest`	`set` and `sorted` (8), `x if cond else y` (8)
`rotate`	`%` (2), slicing and `+` to join two lists (6)
`remove_duplicates`	the "seen" set pattern (8)
`pairwise_diff`	the `zip` neighbours trick (4)
`fizzbuzz`	`range(1, n + 1)` (4), `%` (2), `str()` (8)
`digit_sum`	`abs`, `str`, `int`, `sum(... for ...)` (8)
`is_prime`	a `while` loop (4), `%` (2)
`collatz_steps`	a `while` loop (4), `//` and `%` (2), `raise` (8)
`int_to_base`	`divmod` (8), indexing into the string `"0123456789abcdef"`, `reversed` and `join` (8)
`ex03_fix_me`	`sum`, `len`, `min`, `max`, `lower`, `split`, `join` (8), `range` (4), slicing (6)

Run them from the course root:

```
python run.py 1           # all of today's tests
python run.py 1 -k ex03   # only the tests whose name contains "ex03"
```

A note on ex02: "base conversion" means writing a number in a different base. We normally write numbers in base 10 (digits 0–9), but computers also use base 2 (binary, digits 0 and 1) and base 16 (hexadecimal, digits 0–9 and a–f). For example, 13 in binary is `1101`, because 13 = 8 + 4 + 1. You find the digits by repeatedly dividing by the base and collecting the remainders with `%` and `//`. The docstring explains exactly what's expected. For `ex03_fix_me.py`: **don't rewrite the functions.** Run the tests, read the failure, find the single wrong line, fix it. That's the exam skill. Time yourself: aim for < 3 min per bug. Why not just rewrite them? In the exam the code is long and its parts depend on each other. A rewrite can easily change some behaviour a test relies on and break things that were working. A one-line fix aimed at one specific failing test is faster and much safer.

End-of-day checklist

Tick every box

- [ ] I can predict the output of `-7 // 2`, `a[-0:]`, `[[0]*2]*2` mutation.
- [ ] I know `sort()` returns `None` and `sorted()` returns a new list.
- [ ] I can read a unittest failure and tell “got” from “expected”.
- [ ] All day-1 tests pass.

Exam lens

- Read the failing assertion first; reproduce its input in the REPL, then fix the smallest line that explains it.
- `return` and mutability bugs are common. Check them before digging into deeper logic.
- The tests are the spec: exact types, formats, and boundary behaviour all score points.

Tonight’s bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.